Boot Controller Architecture & Data Flow

SONIC-MCU

Memory Formatting and Data Flow | SPEC-BOOTFLOW-v3.0

Introduction

The SONIC-MCU utilizes a dedicated hardware state machine to handle the bootstrapping process, circumventing the latency and complexity of a software-based Boot ROM. This Bootloader orchestrates an end-to-end Direct Memory Access (DMA) transfer.

It fetches the firmware image from a passive external SPI Flash chip, coordinates the transaction through an internal APB SPI Master IP, applies necessary endianness corrections, and autonomously writes the formatted instructions and data directly into the system's active I-SRAM and D-SRAM via the AXI-Lite crossbar.

Firmware Image and Memory Formatting

The firmware integration pipeline is a critical stage. It bridges the gap between how a standard C-compiler outputs memory arrays and how a physical SPI Flash chip expects data to be organized.

Original Word-Addressed Formats (`ebreak_imem.mem` / `ebreak_dmem.mem`)

The compiler initially outputs the compiled C-code into two distinct memory files for instructions and data. These files are formatted strictly as **32-bit, word-addressed hexadecimal files**.

Listing 1: Snippet from `ebreak_imem.mem` (Word-Addressed)

```
@00000000
00000097
@00000001
13408093
```

In this format, the "`@`" symbol denotes the 32-bit word boundary. Because RISC-V is a Little-Endian architecture, the 32-bit instruction `0x00000097` theoretically translates to four consecutive bytes in memory: `0x97`, `0x00`, `0x00`, `0x00`. However, standard SPI Flash arrays operate on a contiguous, 8-bit byte-addressed space, making this native format incompatible with a raw serial flash dump.

The Conversion Pipeline (`spi_unified_mem_instr_data.py`)

To resolve this, the build flow invokes a Python conversion script. This script executes three vital formatting transformations:

1. **Address Translation:** It maps word addresses to byte addresses (e.g., Word Address `0x00000001` logically maps to Byte Address `0x00000004`). It aggressively pads any skipped addresses in the input file with `00` to guarantee a 1:1 physical layout.
2. **Little-Endian Byte Unpacking:** It intercepts the 32-bit hex strings and decomposes them into Little-Endian byte arrays. The word `13408093` is sliced and rearranged as `93`, `80`, `40`, `13`.
3. **Unified Concatenation:** It merges the converted Instruction memory and Data memory into a single, continuous block, eliminating the need for complex multi-file handling during flash programming.

The Combined Output (`ebreak_combined.mem`)

The result of the script is the final flash image file. It lacks "`@`" address tags because it operates implicitly: line N corresponds exactly to byte address N in the SPI flash memory space.

Listing 2: Snippet from `ebreak_combined.mem` (Byte-Addressed, Little Endian)

```
97
00
00
00
93
80
40
13
```

One-Time Programmable Formatting (`s25fs512sOTP.mem`)

Conversely, internal hardware simulation models (like the `S25fs512s` simulation model) utilize specialized memory maps for their One-Time Programmable (OTP) security registers. The `s25fs512sOTP.mem` file is structured distinctly to interface with the simulator's internal loading routines:

Listing 3: Snippet from `s25fs512sOTP.mem`

```
@000
00
01
02
@010
9F
```

While this file is also byte-oriented, the simulator's `$readmemh` parser expects explicit byte-address anchors (e.g., `@000`, `@010`) to map these isolated security bytes directly into internal status registers rather than the main sequential flash array.

Important

Necessity of the OTP File: The OTP memory contains critical configuration bytes, non-volatile lock bits, and device identification parameters. In physical hardware, these are programmed at the factory or during device provisioning. In the verification environment, the `s25fs512s` simulation model strictly requires the `s25fs512sOTP.mem` file to preload these internal states. If this file is omitted or incorrectly formatted, the flash model will initialize in an undefined or locked state, causing it to reject incoming SPI read commands or assert false protection faults, leading to simulated boot failures.

Boot Controller Finite State Machine

The Bootloader operates as the master conductor of the system immediately out of reset. It is defined by a highly robust data-pump Finite State Machine (FSM).

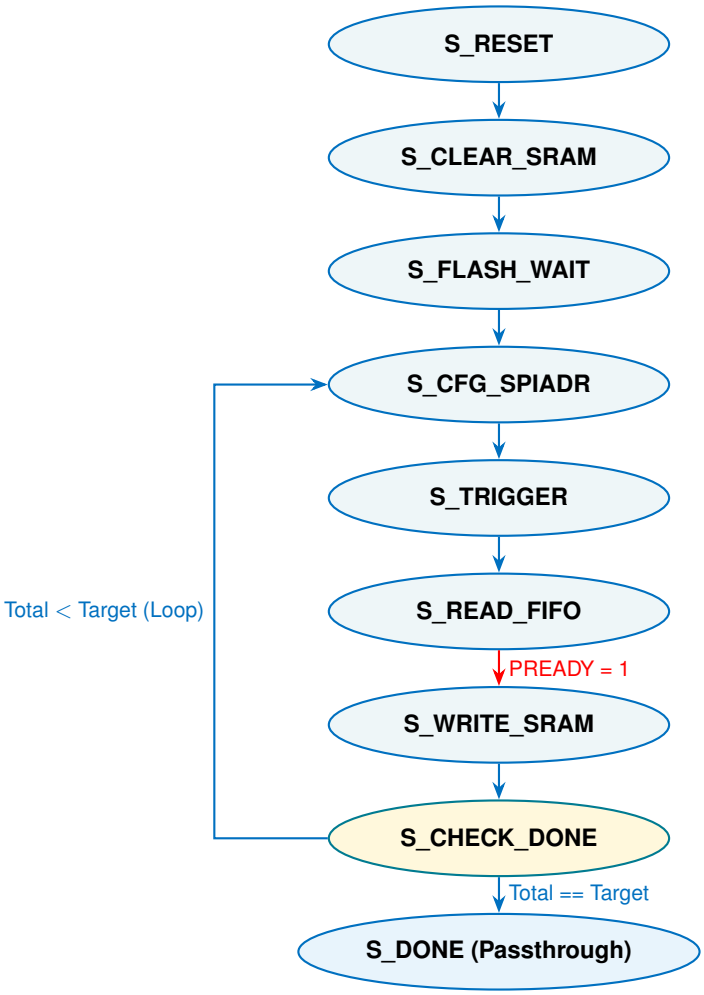


Figure 1: Bootloader DMA Logic Flow

Detailed FSM State Breakdown

- `S_RESET`**:** Evaluates the type of reset. If it's a warm boot (`soft_rst_ni` toggled), it jumps directly to `S_DONE` to preserve SRAM state. If it's a cold boot, it transitions to initialization.
- `S_CLEAR_SRAM`**:** Iterates through the entire instruction and data memory bounds, driving `0x00000000` over the AXI-Lite bus to proactively sanitize the SRAM arrays and prevent X-states.
- `S_FLASH_WAIT`**:** Configures the SPI Master clock divider and waits for a hardcoded number of cycles to satisfy the external Flash chip's power-on/reset recovery time.
- `S_CFG_SPIADR` **(Configure SPI Address):** Before the flash chip outputs data, it requires an instruction. This state packs the 8-bit SPI Read Opcode (`0x03`) and the 24-bit target flash memory address into a single 32-bit word. It issues an APB Write to push this command packet into the SPI Master's **TX FIFO**, priming the controller for the transfer.
- `S_TRIGGER`**:** Issues an APB write to the SPI Master's control register to actively toggle the Chip Select and begin the physical serial clock (`SCK`).
- `S_READ_FIFO`**:** The FSM attempts to read the SPI Master's `RX_FIFO` via APB. Because the serial transfer takes time, the APB bus stalls (pulls `PREADY` LOW). The Bootloader safely sleeps in this state until data arrives.
- `S_WRITE_SRAM`**:** Once `PREADY` asserts and the data is captured, the FSM corrects the endianness (byte-swapping the payload) and fires the formatted 32-bit word over the AXI-Lite interface to the target SRAM address.

- S_CHECK_DONE**:** Increments the internal address and word counters. If the total fetched words equals the target firmware size, it proceeds. Otherwise, it loops back to S_CFG_SPIADR for the next word.
- S_DONE**:** The terminal state. The bootloader asserts boot_done_o, yielding control of the APB bus (enabling passthrough) and releasing the RISC-V core reset, allowing the CPU to boot from 0x0.

Secondary FSM: The SPI Master DMA

While the Bootloader manages the macroscopic transfer logic, it relies entirely on a secondary, autonomous FSM operating inside the apb_spi_master IP to handle the microscopic serial timing.

Once the Bootloader primes the TX FIFO and triggers the transfer (in S_TRIGGER), the SPI Master FSM wakes up from IDLE. It transitions into a SHIFT phase, seizing control of the physical SPI pins. It autonomously toggles the SCK line exactly 32 times, shifting out the command/address from its TX FIFO on the MOSI line, and shifting in the payload from the flash on the MISO line.

Crucially, the RTL of the SPI Master is hardcoded with a 5-bit counter (counter[4:0] == 5'b11111). The instant this counter reaches 31, it raises an End-Of-Transfer flag, transitions to a PUSH state, and dumps the assembled 32-bit word into the RX_FIFO. This push action is what ultimately signals the APB bus to wake up the sleeping Bootloader FSM.

End-to-End Data Flow Pipeline

The journey of a single 32-bit firmware word follows this physical and logical route from the non-volatile storage to the active system memory:

1. **Stage 1: Flash to SPI Master (Physical Layer):** The external Flash chip shifts data out serially on its SO pin. This enters the SoC via the SPI Master's spi_sdi1_w pin. The Master captures 32 bits and pushes them to its APB RX_FIFO.
2. **Stage 2: SPI Master to Bootloader (APB Protocol):** The Bootloader, stalled in the S_READ_FIFO phase, sees the PREADY signal go HIGH. It captures the data from the FIFO across the APB bus.
3. **Stage 3: Endianness Correction & AXI-Lite Target:** Because the data was packed byte-by-byte into the flash (Byte 0 first), it must be reversed. The Bootloader swaps the byte lanes ([7:0] to [31:24], etc.), reconstructing the native Little-Endian format. It is then placed on the AXI-Lite WDATA channel, alongside the target SRAM address on the AWADDR channel, routing the packet directly into the destination memory.

Hardware Synchronization Timing (Waveform)

The waveform below details the timing relationship across the independent buses. It proves how the Boot Controller's APB read transaction is safely suspended, absorbing the slow SPI transaction time natively, and immediately responding the cycle the data arrives.

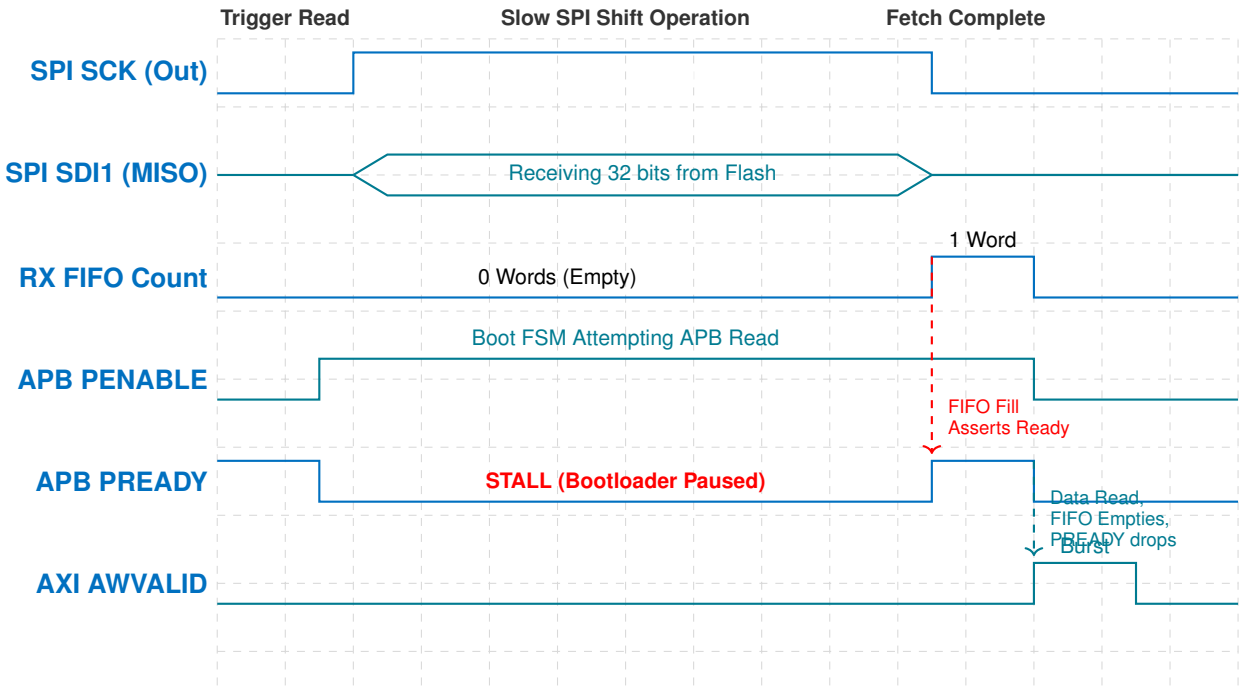


Figure 2: Cycle-Accurate Hardware Timing Sequence

Note

Crucially, when the Bootloader successfully reads the APB bus at cycle 12, it drains the `RX_FIFO`. As the FIFO Count immediately falls back to 0, the SPI Master's internal bus logic concurrently drives `PREADY` back to 0, safely clearing the bus for the next transaction and preventing accidental double-reads.

RTL Implementation & Code Flow

The bootloader is implemented in SystemVerilog utilizing a standard multi-process Finite State Machine architecture. The logic is distinctly separated into sequential state updates, combinational next-state decoding, and active bus driving blocks.

State Register and Reset Handling

The sequential block is clocked on the positive edge of the system clock and handles asynchronous resets. Crucially, it evaluates both `hard_rst_ni` and `soft_rst_ni` to determine whether the system is undergoing a cold or warm boot.

Listing 4: FSM State Register Update

```
always_ff @(posedge clk_i or negedge hard_rst_ni or negedge soft_rst_ni) begin
  if (!hard_rst_ni) begin
    state_q <= S_RESET;      // Cold boot: start from scratch
    // Reset internal counters...
  end else if (!soft_rst_ni) begin
    state_q <= S_DONE;       // Warm boot: bypass SPI, jump to DONE
    // Preserve counters...
  end else begin
    state_q <= state_d;      // Normal operation: transition to next state
    // Update counters...
  end
end
```

Next State Logic (Combinational FSM)

The transition logic dictates how the bootloader advances through its pipeline. It continuously monitors the handshakes from both the APB and AXI domains (e.g., `pready`, `aw_ready`, `w_ready`) to ensure it does not proceed until transactions are properly acknowledged by the external bus structures.

Listing 5: Combinational Next-State Evaluation

```
always_comb begin
  state_d = state_q; // Default: hold state

  case (state_q)
    S_CFG_SPIADR: begin
      // Wait until the APB write command is accepted
      if (apb_mst_resp_i.pready) state_d = S_TRIGGER;
    end
    S_TRIGGER: begin
      if (apb_mst_resp_i.pready) state_d = S_READ_FIFO;
    end
    S_READ_FIFO: begin
      // Stall here until the SPI IP pushes data and asserts PREADY
      if (apb_mst_resp_i.pready) state_d = S_WRITE_SRAM;
    end
    S_WRITE_SRAM: begin
      // Wait for AXI crossbar to accept the write burst
      if (axi_aw_ready && axi_w_ready) state_d = S_CHECK_DONE;
    end
    // ... (other states omitted for brevity)
  endcase
end
```

Bus Interface and Endianness Routing

The output logic actively drives the APB and AXI buses based on the current state. When reading from the SPI `RX_FIFO` (`S_READ_FIFO`), the hardware captures the raw serial payload. Before writing this data to the SRAM (`S_WRITE_SRAM`), the RTL manually restructures the bytes to bridge the Endianness gap between the SPI Flash format and the RISC-V CPU execution format.

Listing 6: Endianness Correction and AXI Routing

```
// Capture data from APB when PREADY asserts
if (state_q == S_READ_FIFO && apb_mst_resp_i.pready) begin
  // Perform Little-Endian byte-swap natively in hardware
  rxfifo_data_q <= { apb_mst_resp_i.prdata[7:0],
                     apb_mst_resp_i.prdata[15:8],
                     apb_mst_resp_i.prdata[23:16],
                     apb_mst_resp_i.prdata[31:24] };
end

// Drive AXI-Lite Write Data Channel
assign axi_req_o.w.data = rxfifo_data_q;
assign axi_req_o.w_valid = (state_q == S_WRITE_SRAM);
```

Boot Performance & Hardware Metrics

During physical RTL verification (`xrun.log` dated Jul 24, 2026), cycle-accurate performance measurements were captured to validate the hardware's temporal efficiency.

The testbench operates on a system clock period of $T_{clk} = 10$ ns (100 MHz base frequency). The APB SPI Master Clock Divider (`SPI_CLK_DIV`) is parameterized to a constant value of **4**, generating a 12.5 MHz physical SPI Serial Clock. The firmware footprint utilized for the benchmark was exactly **6,156 bytes** (1,539 words).

Table 1: Cycle-Accurate Boot Latency Breakdown

Boot Phase	Clock Cycles	Absolute Time
Configuration Phase	35,135	351.34 μs
SRAM Feeding Phase	989,583	9.89 ms
Total Boot Latency	**1,024,718**	**10.24 ms**

- **Configuration Phase:** Encompasses the `S_CLEAR_SRAM` pass (zeroing out the entire 2,560 words of target memory bounds to prevent X-states) and the `S_FLASH_WAIT` power-up stall mandated by the vendor's physical flash simulation model (30,000 cycles).
- **SRAM Feeding Phase:** The active DMA transfer window fetching all 1,539 words sequentially across the SPI bus and dispatching them via AXI-Lite.
- **Absolute Determinism:** The decoupled hardware FSM architecture guarantees absolute temporal determinism. Repeating the cold boot sequence continuously yields an identical footprint of exactly 1,024,718 cycles without deviation.